Distributed Key-Value Store (HLD) Interview Guide

Example Use Case: E-commerce Add-to-Cart

1. Problem Statement

Design a highly available, horizontally scalable, distributed key-value database for an Add-to-Cart service. Goals: scalability, decentralization, high availability, and eventual consistency.

2. Data Model

Key = UserID. Value = Cart (ProductID -> Quantity). APIs: PUT(Add/Update), GET(Fetch Cart), DELETE(Remove Item).

3. High-Level Approach

Step 1: Partition data using Consistent Hashing.

Step 2: Replicate each key on N replicas (Preference List).

Step 3: Route GET/PUT through a Coordinator.

Step 4: Use Quorums (R, W, N).

Step 5: Resolve conflicts using Vector Clocks.

Step 6: Exchange cluster metadata using Gossip Protocol.

Step 7: Repair replicas using Merkle Trees.

4. Partitioning

Why? A single machine cannot store all keys.

Use Consistent Hashing to distribute keys across servers.

Advantage over hash(key)%N: adding/removing servers moves only neighboring keys.

Virtual Nodes improve load balancing.

5. Replication

Replication Factor N stores copies on multiple servers.

Coordinator determines the preference list and forwards requests.

Benefits: fault tolerance, availability and read scalability.

6. GET / PUT and Quorum

PUT writes to replicas and succeeds after W acknowledgements.

GET reads from R replicas.

Choose R + W > N to ensure read/write quorum overlap.

Typical values: N=3, R=2, W=2.

7. Data Versioning

Replicas may diverge during failures.

Vector Clocks record causal history instead of timestamps.

Concurrent versions are returned for application-level conflict resolution.

This enables eventual consistency without losing updates.

8. Gossip Protocol

Each node periodically exchanges metadata with random peers.

Eventually every node knows:

• Live/failed nodes

• Partition ownership

• Coordinator information

No centralized metadata server is required.

9. Merkle Trees

Each replica stores hashes in a tree.

During synchronization compare root hashes.

Only mismatching subtrees are traversed and synchronized.

Efficient anti-entropy repair.

10. Typical Interview Answer Flow

1. Explain use case and data model.

2. Mention goals.

3. Explain partitioning.

4. Explain replication.

5. Describe GET/PUT.

6. Explain quorum.

7. Explain vector clocks.

8. Explain gossip.

9. Explain Merkle trees.

10. Summarize trade-offs (Availability + Partition Tolerance, Eventual Consistency).

11. Common Interview Questions

Q: Why Key-Value DB?

A: Single-key access, O(1)-style lookups, easy horizontal scaling.

Q: Why partition by UserID?

A: Keeps entire cart on one shard.

Q: Why Consistent Hashing?

A: Minimal key movement during scaling.

Q: What are Virtual Nodes?

A: Multiple logical positions per physical server for balanced load.

Q: Why Replication?

A: High availability and fault tolerance.

Q: Explain R+W>N.

A: Read and write quorums overlap, improving freshness.

Q: Why Eventual Consistency?

A: Better availability and lower latency.

Q: Why not timestamps?

A: Clock skew; vector clocks capture causality.

Q: What does Gossip solve?

A: Decentralized membership and failure detection.

Q: What do Merkle Trees solve?

A: Efficient replica synchronization.

Q: What happens if a replica fails?

A: Writes continue if quorum is met; repair occurs later.

Q: CAP theorem?

A: Dynamo favors Availability and Partition Tolerance.

12. Key Takeaways

• Partition for scalability.

• Replicate for availability.

• Quorums balance consistency and latency.

• Vector clocks detect conflicting writes.

• Gossip keeps cluster metadata synchronized.

• Merkle trees repair replicas efficiently.

• Together these form the foundation of Amazon Dynamo-style databases.